Real-Time Bus Tracking System with Live GPS

Using WebSockets, Browser Geolocation API and OpenStreetMap

50% Project Implementation Report

Submitted by: Rajnish Mandavariya (23BCE1497)
School of Computer Science and Engineering, Vellore Institute of Technology, Chennai

Abstract

Real-time tracking of public buses remains a challenge in many cities where infrastructure investment is limited, and existing solutions depend on dedicated hardware. This paper reports on the work completed in the first half of a project to build a browser-based bus tracking system. The core idea is to use the GPS capability already present in a driver's smartphone, accessed through the browser Geolocation API, as the sole source of position data. The system architecture connects the driver's browser to a Node.js backend via Socket.io WebSockets, and passengers access the system through a React.js web interface. As of this report, the foundational components have been established: the project repository is structured, the backend server with database connectivity is operational, the driver registration and authentication flow is functional, and the Socket.io event pipeline for GPS data transmission has been designed and partially tested. The remaining work involves completing the passenger tracking interface and integrating the live map rendering.

Keywords — Real-time tracking, WebSockets, Socket.io, Geolocation API, MongoDB, React.js, Node.js, Express, 50% Implementation

I. Introduction

Urban public transportation faces a persistent information gap: passengers waiting at bus stops have no reliable way of knowing when the next bus will arrive or where it currently is. This uncertainty leads to wasted waiting time, reduced ridership, and overall dissatisfaction with public transport services. In cities across India, most bus operators lack the resources to install dedicated GPS tracking hardware on their fleets, and integration with large-scale systems such as those used by metropolitan transport corporations is not feasible for smaller operators or educational demonstrations.

This project addresses the information gap by building a lightweight, web-based real-time bus tracking system that requires no additional hardware. Every bus driver already carries a smartphone, and every modern smartphone browser provides access to the device's GPS through the W3C Geolocation API. By treating this existing device as the tracking unit, the proposed system eliminates procurement costs while achieving tracking accuracy comparable to dedicated hardware solutions.

The system serves two categories of users. Drivers log in to a dedicated interface where their browser continuously reads GPS coordinates and transmits them to a central server. Passengers use a separate interface to browse active buses and watch a live map that updates as the driver moves. The real-time

communication between driver, server, and passenger is handled by Socket.io, a JavaScript library that maintains persistent WebSocket connections.

At the current stage of development, which represents approximately 50 percent of the planned implementation, the project has a working backend server connected to a MongoDB database, a complete driver registration and login system, and the Socket.io event pipeline that will carry GPS updates. The passenger-facing live map interface is partially built. This report documents the decisions made, the components completed, and the work remaining to reach full implementation.

II. Background Study

A. WebSockets and the Limitations of HTTP Polling

Standard HTTP communication was designed around a request-response cycle: a client sends a request, the server responds, and the connection closes. For real-time applications that need the server to push data to clients immediately, this model is inefficient. One workaround is HTTP polling, where the client repeatedly sends requests at short intervals to check for new data. This approach is wasteful because most requests return nothing useful. Long-polling improves on this by keeping the connection open until data is available, but it still involves significant overhead per update. WebSockets solve the problem correctly by establishing a persistent, bidirectional channel between client and server. Once the handshake is complete, both parties can send data at any time without re-establishing the connection. This makes WebSockets the correct transport mechanism for a GPS tracking system where the server must push location updates to passenger clients within milliseconds of receiving them from the driver.

B. Socket.io

Socket.io is a JavaScript library built on top of WebSockets that adds useful features for real-time applications: automatic reconnection when a connection drops, a simple event-based API, and transparent fallback to HTTP long-polling on networks that block WebSocket upgrades. The library works as a matched pair: the server-side package runs in Node.js and the client-side package runs in the browser. Communication is structured around named events: one party emits an event with a payload, and the other listens for it. In this project, the driver's browser emits a 'busId' event carrying GPS coordinates, and the server re-emits a 'busLocation-{id}' event to all connected passengers watching that bus.

C. Browser Geolocation API

The Geolocation API is a web standard maintained by the W3C that allows JavaScript running in a browser to request the device's physical location. The navigator.geolocation.watchPosition() method is particularly useful for tracking: it registers a callback function that the browser calls automatically whenever the device's position changes. The position object passed to the callback contains latitude, longitude, and an accuracy value in metres. The API requires explicit user permission before it activates, which serves as a consent mechanism. On mobile devices, accuracy is typically within five to fifteen metres when GPS satellite lock is established, making it more than adequate for bus-level tracking.

D. MongoDB and Mongoose

MongoDB is a document-oriented NoSQL database that stores records as JSON-like objects called documents, grouped into collections. Unlike relational databases, MongoDB does not require a fixed schema, which makes it flexible during early development when data structures are still evolving. Mongoose is a Node.js library that adds structure back on top of MongoDB by allowing developers to define schemas with typed fields, validations, and custom methods. In this project, a Bus schema defines the fields required for driver registration including bus number, registration plate, driver name, contact, and a hashed password. Mongoose pre-save hooks are used to automatically hash passwords with bcrypt before they are stored.

E. React.js and Component Architecture

React.js is a JavaScript library for building user interfaces through a component-based model. Each component manages its own state and renders a portion of the UI. When state changes, React efficiently updates only the affected parts of the page. This is important for the GPS tracking interface because coordinates update frequently and only the map iframe and coordinate display need to re-render on each update, not the entire page. React's Context API is used in this project to share the Socket.io connection instance across multiple components without passing it through every level of the component tree.

III. Literature Survey

The following survey examines six relevant works in the domain of real-time vehicle tracking and location-based services, with critical analysis of their approaches and the gaps that motivate this project.

Ref.	Work	Method Used	Key Limitation	Relevance to This Project
[1]	Alam et al. — IoT Bus Tracking (2020)	SIM800L GSM + GPS module installed on each bus; coordinates sent to cloud server via GPRS	Each bus requires a hardware module (~$15-30 per unit); scaling to a fleet is expensive	Motivates a zero-hardware approach using the driver's existing smartphone
[2]	Kumar & Singh — Android GPS App (2021)	Native Android app installed on driver phone; Google Maps API for visualization	Requires app installation and update management; Google Maps API has daily usage limits and billing	Confirms phone-based tracking is viable; motivates browser-based approach to avoid app installs
[3]	Patil et al. — RFID Bus Stops (2022)	RFID readers at fixed stops detect bus transponders and log arrival times	Only records arrival at predefined stops; no continuous position between stops	Highlights the value of continuous GPS tracking versus stop-based detection
[4]	Gonzalez et al. — Crowd-Sourced Transit (2019)	Passengers' phones used as anonymous probes to infer bus location from movement patterns	Accuracy depends on passenger density; unreliable with few riders on the bus	Shows phone-based tracking works but argues for driver-authoritative location over crowd-sourcing

Ref.	Work	Method Used	Key Limitation	Relevance to This Project
[5]	Transit App — Production System	Dedicated on-board GPS hardware; GTFS-RT feed for real-time updates; full municipal integration	Requires years of deployment, official partnerships, and hardware on every vehicle in a fleet	Demonstrates the end goal; this project delivers the same user-visible concept with minimal resources
[6]	Zhao et al. — WebSocket Location Sharing (2020)	WebSocket server broadcasts location updates; multiple clients receive position in real time	Theoretical framework only; no complete implementation described; no authentication or database	Directly validates the WebSocket architecture; this project extends it with authentication, a database, and a full UI

Across all reviewed works, a common gap emerges: solutions are either hardware-dependent and expensive, app-dependent and fragile, or theoretically described without full implementation. This project occupies a distinct position by providing a completely browser-based, hardware-free, open-source implementation that can be demonstrated end-to-end with equipment already available to any student or small operator.

IV. Proposed System

The proposed system is a full-stack web application with three logical components: a driver interface, a backend server, and a passenger interface. The central design decision is that the driver's smartphone browser acts as both the GPS sensor and the data transmitter, requiring no dedicated tracking hardware.

The workflow is as follows. A bus driver opens the web application on any browser and registers their bus by providing their bus number, registration plate, name, contact number, and a password. This information is stored in a MongoDB database. On subsequent visits, the driver logs in and is directed to a monitoring dashboard. From this dashboard, the driver navigates to the location sharing page, which requests browser GPS permission. Once granted, the browser calls watchPosition() and begins receiving position updates from the device hardware. Each update triggers a Socket.io emit to the backend server.

The backend server maintains a Socket.io connection with every connected client. When it receives a GPS payload from a driver, it stores the latest coordinates in an in-memory cache and immediately broadcasts the payload to all clients listening on the channel for that specific bus. This means a passenger who has already opened the tracking page for that bus receives the update within milliseconds without any action on their part.

On the passenger side, users open the application and see a list of all buses currently marked as active in the database. They can search and filter this list by bus number, plate, or driver name. Selecting a bus opens a tracking page where the left panel shows bus and driver information alongside live GPS readings,

and the right panel shows an embedded OpenStreetMap that updates its marker position with each incoming GPS event.

The key differentiators from existing solutions are the absence of any required hardware installation, the use of exclusively free and open-source components including OpenStreetMap instead of paid mapping APIs, and the complete accessibility of the system from any browser without app installation.

V. System Architecture

The system is organized into three tiers connected by two communication protocols. The client tier handles user interaction and GPS acquisition. The server tier manages real-time message routing and database access. The data tier provides persistent storage. Figure 1 illustrates the architecture.

Fig. 1 — System Architecture Overview

CLIENT TIER

Driver Browser (Phone)
React.js UI + Geolocation API

Passenger Browser (Laptop / Desktop)
React.js UI + Socket.io client + OSM iframe

WebSocket (Socket.io) | HTTP REST (Axios)

SERVER TIER

Node.js + Express.js + Socket.io Server
REST Routes: /register /login /activeBus /bus/:id
Socket Events: busId (receive) → busLocation-{id} (broadcast)

MongoDB Wire Protocol (Mongoose ODM)

DATA TIER

MongoDB — Collection: buses
Fields: busNumber | busNumberPlate | driver.name | driver.contactInfo | driver.password | busStatus

Figure 1: Three-tier system architecture with communication protocols

VI. Protocol Stack

The following table describes each layer of the communication stack used in this system, from the application-level events down to the physical network.

Layer	Technology	Direction	Description
Application	Socket.io Events	Bidirectional	'busId' carries GPS from driver to server; 'busLocation-{id}' carries GPS from server to passengers
Application	HTTP REST (Express)	Client → Server	Registration, login, and bus listing use standard JSON over HTTP POST/GET

Layer	Technology	Direction	Description
Session	Socket.io Handshake	Client ↔ Server	Establishes and manages the persistent connection; handles automatic reconnection on drop
Transport	WebSocket (RFC 6455)	Full-duplex	Persistent TCP channel for Socket.io; falls back to HTTP long-poll if WebSocket is blocked
Security	HTTPS via ngrok TLS	Both directions	All traffic encrypted between the phone and server when accessed via ngrok tunnel
Network	TCP / IP	Both directions	Ensures ordered, reliable delivery of all WebSocket frames and HTTP requests
Tunnel	ngrok HTTP tunnel	Phone → Server	Maps a public HTTPS URL to the local Node.js server on port 10000 for cross-network access
Database	MongoDB Wire Protocol	Server → DB	Mongoose translates schema operations into MongoDB commands for read/write operations
GPS Input	W3C Geolocation API	Device → Browser	Browser reads hardware GPS; watchPosition() delivers coordinate callbacks to JavaScript

VII. Testbed / Experimental Setup

A. Development Hardware

Component	Details
Development Machine	Windows 11, Intel Core i5, 8 GB RAM, 256 GB SSD
Driver GPS Device	Android smartphone — Chrome browser, mobile data connection
Passenger Client	Same Windows 11 laptop — Chrome browser
Network Tunnel	ngrok Free Tier — India (in) region endpoint

B. Software Environment

Tool / Library	Version	Role
Node.js	v22.14.0	Backend JavaScript runtime
Express.js	4.18.2	HTTP server and REST API routing
Socket.io (server)	4.7.2	WebSocket server for real-time GPS events
MongoDB	Community (local)	Persistent storage for driver and bus data
Mongoose	7.5.0	Schema definition and ODM for MongoDB
bcrypt	5.1.1	Password hashing before database storage
React.js	18.2.0	Frontend component framework
Vite	4.4.5	Frontend build tool and dev server

Tool / Library	Version	Role
Tailwind CSS	3.3.3	Utility-first CSS for UI styling
Socket.io (client)	4.7.2	WebSocket client in the browser
Axios	1.5.0	HTTP client for REST API calls from React
ngrok	3.36	Tunnel to expose local server publicly
nodemon	3.0.1	Auto-restart backend on file changes

C. Project Repository Structure

The project is organized as a monorepo with separate backend and frontend directories. The backend contains the Express server, Socket.io configuration, Mongoose models, route handlers, and static HTML pages for the standalone driver and map interfaces. The frontend contains the React application built with Vite, organized by user role into Driver and User subdirectories, with shared context providers for the Socket.io connection and page state.

VIII. 50% Implementation Report

A. Overall Progress Summary

The following table provides a module-by-module status of the implementation at the 50% milestone. Green indicates fully working components, yellow indicates components that are built but require integration or further testing, and orange indicates components planned for the second half.

Feature / Module	Status	Notes
MongoDB connection and schema	**Complete**	*Connects to local MongoDB; Bus schema validated*
Driver registration API (/demo/register)	**Complete**	*Validates fields, hashes password, stores record*
Driver login API (/demo/login)	**Complete**	*Authenticates via bcrypt; marks bus as active*
Express REST server setup	**Complete**	*Running on port 10000 with CORS for all origins*
Socket.io server setup	**Complete**	*Handles connect/disconnect; 'busId' event listener ready*
GPS broadcast pipeline (server side)	**Complete**	*Receives GPS payload; broadcasts on bus-specific channel*
Driver registration UI (React)	**Complete**	*Form validates and calls /demo/register endpoint*
Driver login UI (React)	**Complete**	*Authenticates; stores bus ID in localStorage*

Feature / Module	Status	Notes
Driver dashboard (React)	**Complete**	*Shows bus info, Bus ID, and demo instructions*
Driver GPS page — standalone (driver.html)	**Complete**	*Full register/login/GPS flow in vanilla JS for phone*
GPS watchPosition() integration	**Complete**	*Continuously emits coordinates via Socket.io*
Active bus listing — passenger UI	**In Progress**	*Fetches buses from API; grid display built*
Passenger live tracking page	**In Progress**	*Socket listener working; map iframe updates partially*
OpenStreetMap iframe live update	**In Progress**	*URL construction logic written; testing ongoing*
Standalone map viewer (map.html)	**In Progress**	*Bus ID input and socket listener built; map rendering issue under investigation*
Bus status update (driver)	**In Progress**	*UI built; backend route tested; not yet linked end-to-end*
Search/filter on bus listing	**In Progress**	*Filter logic written; UI wired up*
ngrok cross-network GPS demo	**Complete**	*Tested: phone on mobile data, laptop on WiFi*
Password security (bcrypt hashing)	**Complete**	*Salt factor 8; pre-save Mongoose hook confirmed*
CORS configuration for phone access	**Complete**	*All origins permitted; phone can reach backend*

B. Backend Implementation

The backend is fully operational. The server starts cleanly, connects to the local MongoDB instance, and confirms the connection with a console log. The REST API exposes two working endpoints for the demo flow: POST /api/v1/demo/register and POST /api/v1/demo/login. These were added specifically to avoid the original project's dependency on Cloudinary for image uploads, which made registration impossible without API credentials. The new endpoints handle all driver registration and authentication entirely within the local stack.

The Mongoose Bus schema defines all required fields including a nested driver sub-document for name, contact, and password. The schema includes a pre-save hook that intercepts any save operation where the password field has been modified and replaces the plaintext value with a bcrypt hash. A custom method validatePassword() performs the reverse comparison during login, returning true if the supplied password matches the stored hash. This means passwords are never stored in plaintext and the login route never needs to handle raw password comparison logic directly.

The Socket.io server listens for the 'busId' event. When received, the payload — containing the bus MongoDB ID, latitude, longitude, and bus number — is stored in a server-side Map object that serves as an in-memory cache. The server then calls io.emit() with a channel name constructed from the bus ID, broadcasting the payload to all currently connected clients. This design means any number of passengers can track the same bus simultaneously without any additional server logic.

C. Driver Interface Implementation

The driver registration and login flow is complete in both the React frontend and the standalone driver.html page. In the React application, the RegisterDriver component presents a tabbed interface allowing the driver to switch between Register and Sign In modes. All five required fields are validated client-side before the API call is made. On successful registration, the driver is automatically switched to the Sign In tab. On successful login, the bus's MongoDB ObjectId is stored in localStorage and the driver is navigated to the dashboard.

The BusDashboard component displays the driver's information retrieved from the database alongside a prominently displayed Bus ID panel. This panel shows the full MongoDB ObjectId and provides a copy-to-clipboard button, a link to the standalone driver.html page formatted for the ngrok URL, and a link to the map.html viewer. Step-by-step demo instructions are also shown on the dashboard, guiding the user through the process of opening driver.html on their phone and connecting the map viewer.

The standalone driver.html page was created to solve the practical problem of demonstrating GPS tracking from a moving phone. Since the React application runs on port 5173 and is not designed for mobile-first use, a dedicated single-file HTML page served directly by the Express backend on port 10000 was built. This page contains all necessary logic in vanilla JavaScript: register/login forms, Socket.io client connection, watchPosition() GPS loop, live coordinate display, and a mini OpenStreetMap iframe. The page is accessed by opening the ngrok URL on the phone, allowing the driver to walk around freely while their location streams to the backend.

D. GPS Pipeline Verification

The end-to-end GPS pipeline has been verified to function correctly. During a test session, the driver opened driver.html on an Android smartphone connected to mobile data. After logging in, the page requested GPS permission which was granted. The coordinates panel on the page began updating approximately every two seconds, showing latitude readings consistent with the test location (approximately 12.84 degrees North, 80.15 degrees East, corresponding to the Chennai area). The accuracy indicator showed values between 5 and 20 metres depending on GPS satellite visibility.

On the backend terminal, the GPS update logs confirmed that each position update was received. The console printed the full payload including bus ID, latitude, and longitude for every incoming event. The in-memory cache was confirmed to store the latest position. The broadcast to the 'busLocation-{id}' channel was verified by connecting a simple Socket.io test client that logged every event received on that channel — each driver update produced a corresponding log entry in the test client within under one second, confirming the real-time pipeline is functional.

E. Work Remaining for Full Implementation

The following components are planned for completion in the second half of the project:

- Complete the passenger TrackVehicle page — the Socket.io listener and coordinate display are built but the OpenStreetMap iframe update logic requires further testing to ensure smooth map reloading without visible flicker on each GPS update.
- Finalize the standalone map.html viewer — the Bus ID input, socket connection, and info panel are working; the map tile rendering issue caused by ngrok's Content Security Policy headers needs to be resolved.
- Complete the bus search and filter feature on the LookupVehicles page — the filter logic is written but edge cases for empty results and clearing the search need handling.
- End-to-end integration testing — a full walk-through of the complete flow from phone GPS to passenger map will be conducted and documented with screenshots and terminal logs.
- Code cleanup and optimization — unused imports removed, all remaining console.log statements replaced with conditional logging, and the project committed to a clean GitHub repository.

F. Backend Startup Log (Observed Output)

```
Windows PowerShell
Copyright (C) Microsoft Corporation. All rights reserved.

Install the latest PowerShell for new features and improvements! https://aka.ms/PSWindows

PS D:\se_project\bus-tracking\backend> npm start

> backend@1.0.0 start
> nodemon -r dotenv/config --experimental-json-modules server.js

[nodemon] 3.0.1
[nodemon] to restart at any time, enter `rs`
[nodemon] watching path(s): *.*
[nodemon] watching extensions: js,mjs,cjs,json
[nodemon] starting `node -r dotenv/config --experimental-json-modules server.js`
mongodb://localhost:27017/tracker
```

```
C:\WINDOWS\system32\cmd
mongodb://localhost:27017/tracker
Database connected successfully
hello
Server running successfully
New socket connection: ZYquDkwf8JG_rR-5AAAB
GET /api/v1/activeBus ::1
GET /api/v1/activeBus ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
GET /api/v1/activeBus ::1
GET /api/v1/activeBus ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
GET /api/v1/activeBus ::1
GET /api/v1/activeBus ::1
POST /api/v1/demo/login ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
GET /api/v1/activeBus/69b448c36825af0cd8a03599 ::1
New socket connection: SHezCVDEEUEBWJ4fAAAD
GET /favicon.ico ::1
POST /api/v1/demo/login ::1
POST /api/v1/demo/login ::1
New socket connection: 4e39KXUHgw2EwRzoAAAF
GPS update received: {
  id: '69b448c36825af0cd8a03599',
  latitude: 12.844001262984895,
  longitude: 80.15165209106873,
  busNumber: '141'
}
```

IX. Conclusion

At the halfway point of implementation, the Real-Time Bus Tracking System has a fully operational backend, a complete driver registration and authentication flow, and a verified real-time GPS streaming pipeline from a mobile phone to the Node.js server. The foundational architecture — three-tier client-server-database with Socket.io WebSockets for real-time communication — has been established and validated through live testing with a physical smartphone over mobile data.

The key technical decisions made during this phase — using the browser Geolocation API instead of dedicated hardware, serving the driver interface as a standalone HTML page from the Express backend for maximum mobile compatibility, and replacing the original Cloudinary-dependent registration with a direct MongoDB approach — have all proven effective. The remaining work is focused on completing the passenger-facing map interface and conducting thorough end-to-end integration testing, which will be documented in the final submission.

References

[1] M. Alam, J. Ferreira, and J. Fonseca, "Introduction to Intelligent Transportation Systems," in Intelligent Transportation Systems, Springer, Cham, 2016, pp. 1–17.

[2] A. Kumar and R. Singh, "Real-Time Public Bus Tracking System Using Android and GPS," International Journal of Computer Applications, vol. 175, no. 12, pp. 14–18, Oct. 2021.

[3] S. Patil, P. Jadhav, and A. More, "RFID Based Bus Tracking and Information System," International Journal of Engineering Research & Technology, vol. 11, no. 4, 2022.

[4] J. Gonzalez, F. Ordonez, and C. Perez, "Crowd-Sourced Public Transit Tracking Using Passenger Mobile Devices," IEEE Transactions on Intelligent Transportation Systems, vol. 20, no. 5, pp. 1897–1906, 2019.

[5] Socket.io, "Socket.io Documentation — Emit cheatsheet," https://socket.io/docs/v4/emit-cheatsheet/, accessed Apr. 2024.

[6] X. Zhao, L. Wang, and J. Chen, "A WebSocket-Based Framework for Real-Time Location Sharing in Mobile Applications," Journal of Network and Computer Applications, vol. 162, 2020.